Memory Management & Garbage Collection

AJAY

TABLE OF CONTENTS - TOC

Two Types of Memory

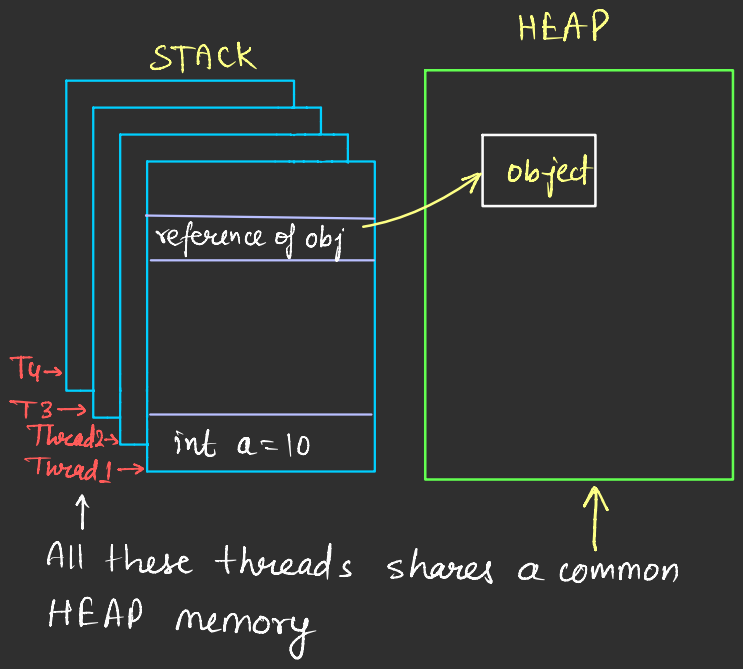
2 Types of memory : → RAM

- STACK
- HEAP

] Both stack & heap are created by JVM & stored in RAM.

• Stack Memory

- Store temporary variables & separate memory block for method
- Store primitive data types
- Store Reference of the heap objects.
  - Strong Reference
  - Weak Reference
    - Soft Reference
- Each Thread has its own stack memory.
- Variables within a SCOPE is only visible and as soon as any variable goes out of the SCOPE, it get deleted from the stack (in LIFO order).
- When stack memory goes full, its throws "java.lang.StackOverflowError"

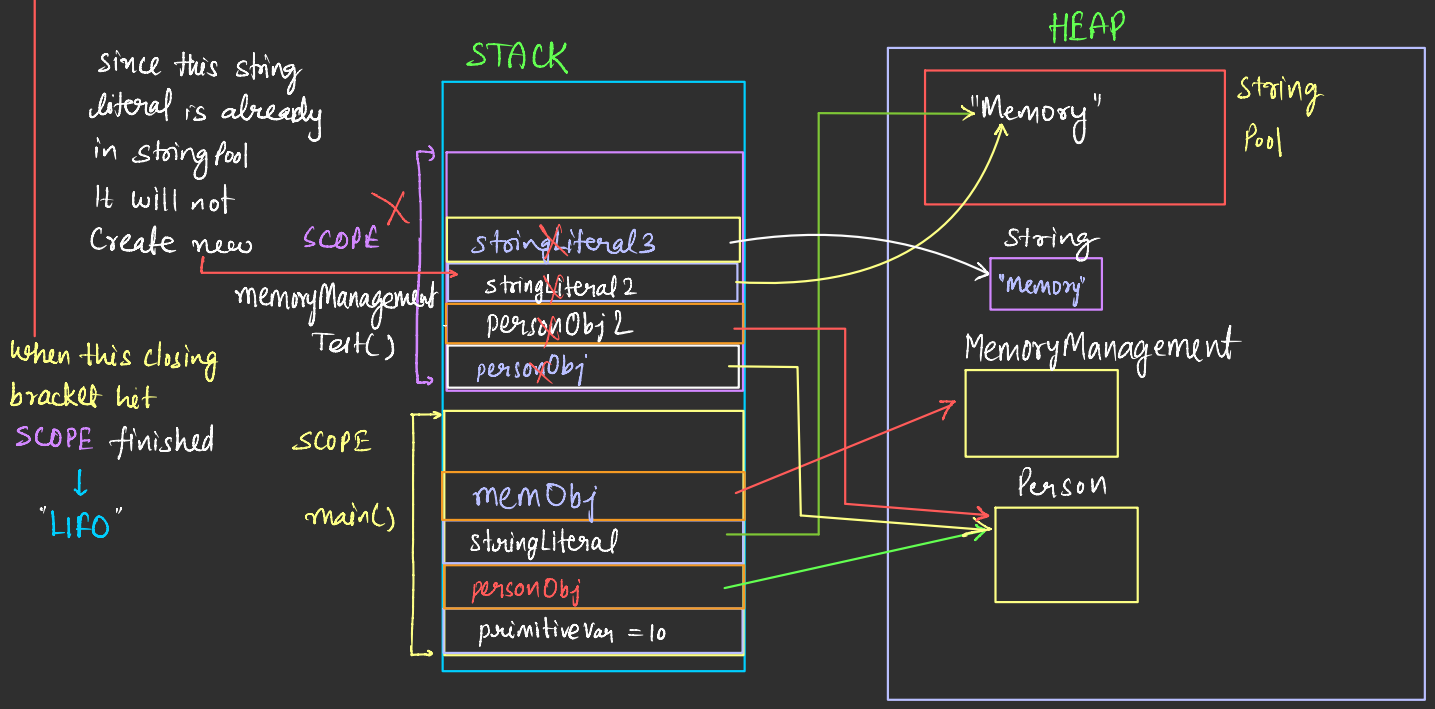


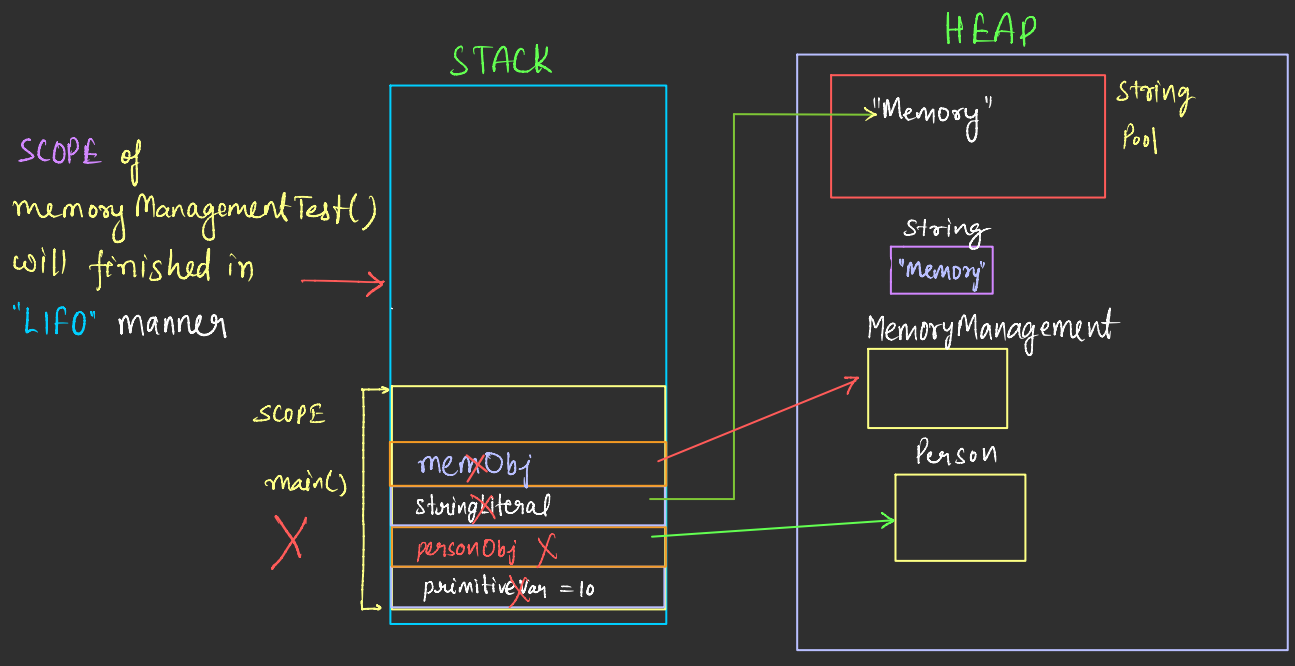
HEAP > STACK

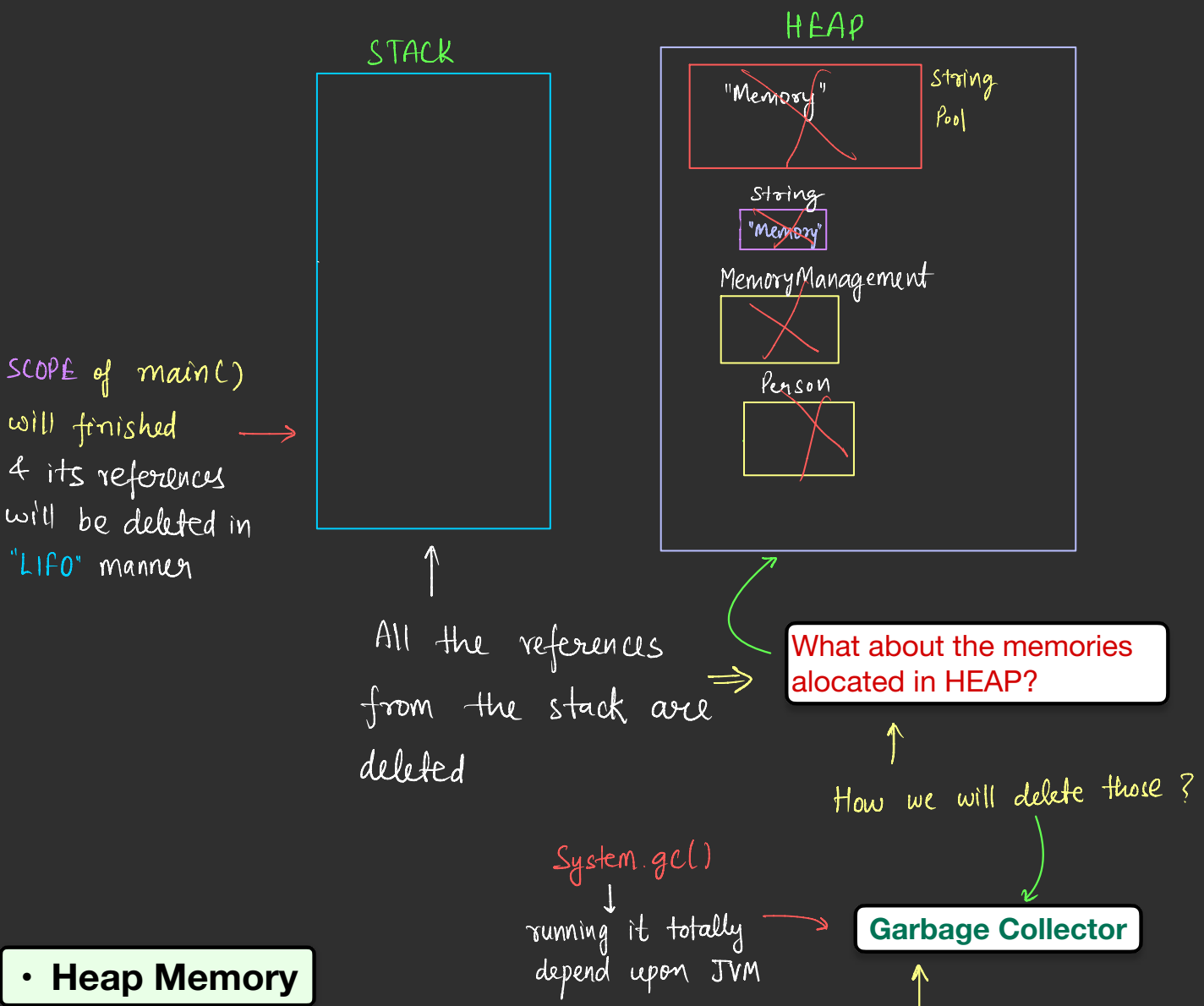
TOC

```java
package com.ajay.javafoundry.core.memoryManagement;

public class MemoryManagement {

    public static void main(String[] args) {

        int primitiveVar = 10;
        Person personObj = new Person();
        String stringLiteral = "Memory";

        MemoryManagement memObj = new MemoryManagement();
        memObj.memoryManagementTest(personObj);
    }

    private void memoryManagementTest(Person personObj) {

        Person personObj2 = personObj;
        String stringLiteral2 = "Memory";
        String stringLiteral3 = new String("Memory");
    }

}
```

Below the stack & Heap memory allocation for the given program.







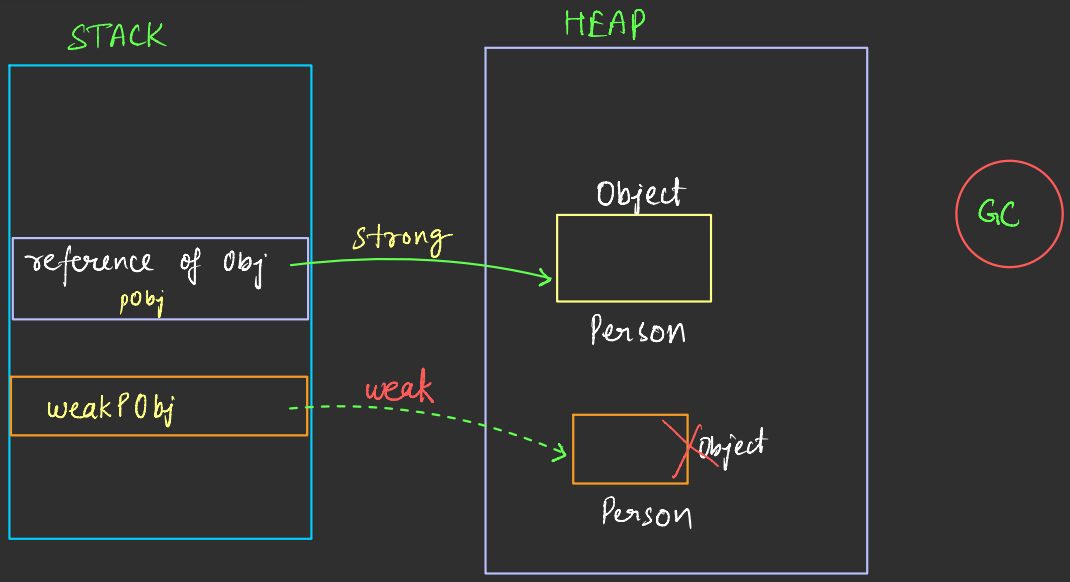
• Heap Memory

- Store Objects
- There is no order of allocating memory
- Garbage Collector is used to delete the unreferenced objects from the Heap.
  - Mark & Swap Algorithm
  - Types of GC
    - Single GC
    - Parallel GC
    - CMS (current Mark Sweep)
    - G1 GC
- Heap memory is shared with all the Threads.
- Heap also contains the string pool
- When heap memory goes full, its throws "java.lang.OutofMemoryError"

- Heap memory is further divided into:
  - Young Generation (Minor GC happens here)
    - Eden
    - Survivor
  - Old / Tenured Generation (Major GC happen here)
  - Permanent Generation – Metaspace
    - ↓ Befor java 7 (was part of heap)
    - ↓ after java 7 (non-heap)

TOC

• Strong and Weak Reference



① Strong Reference ⇒

```
Person pObj = new Person();
```

↳ When GC runs, it look if it is any strong reference there, If it is it will not delete it.

② Weak Reference

↑ Till GC is not run, able to access heap object. When GC runs → free object.

```
WeakReference<Person> weakPObj = new WeakReference<Person>(new Person());
```

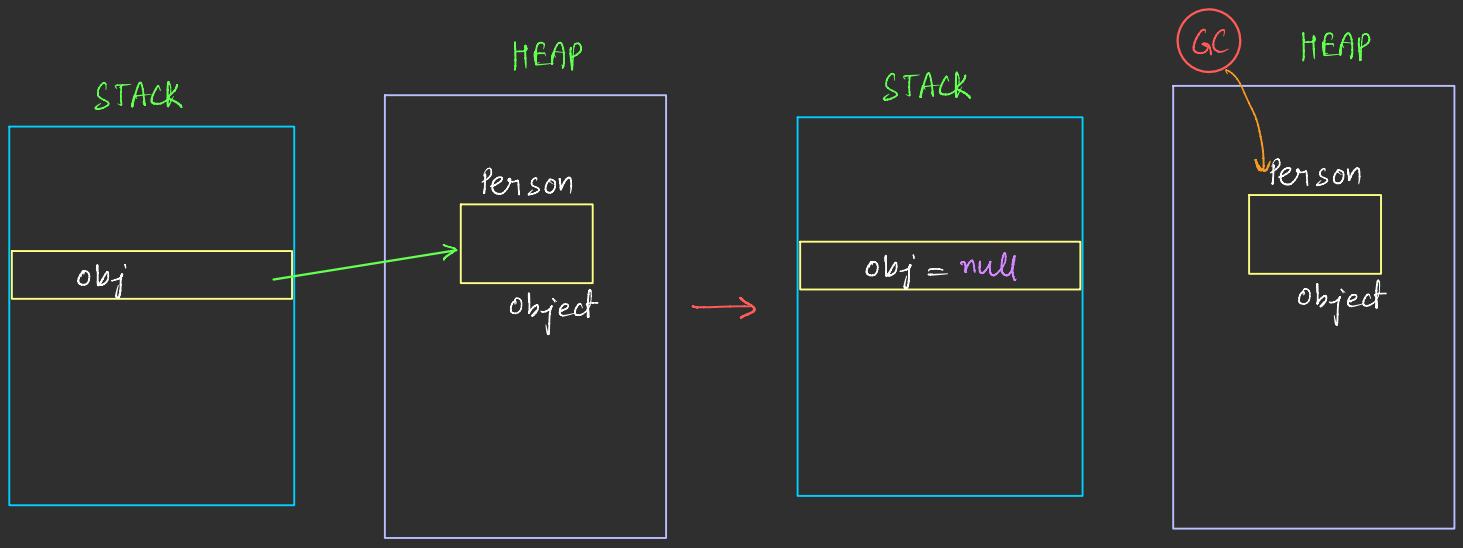
↳ Soft Reference:

- A type of weak Reference
- Tells GC you can delete me But if it is very very urgent.

There is no more space & you need space.

↳ Generally we don't use weak References.

- **When an Object is Garbage Collectable?**



Person obj = new Person();

① obj = null;

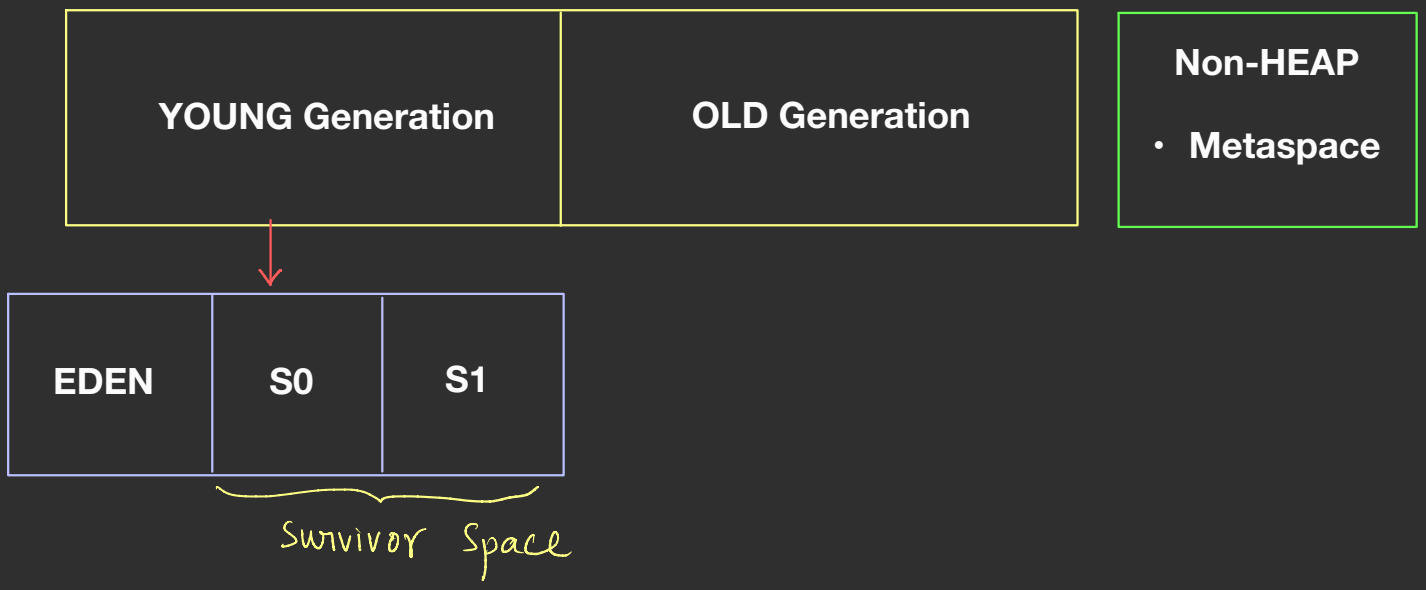
② Person obj1 = new Person();

Person obj2 = new Person();

obj1 = obj2

This is Garbage Collectable

• Structure of Heap Memory



New object goes inside EDEN

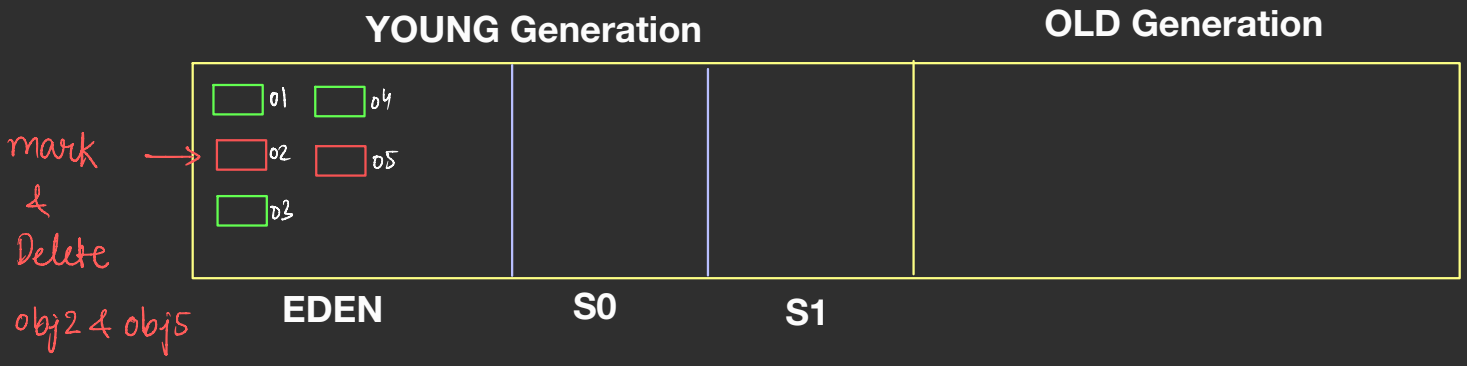
obj 1 obj 4
obj 2 obj 5
obj 3

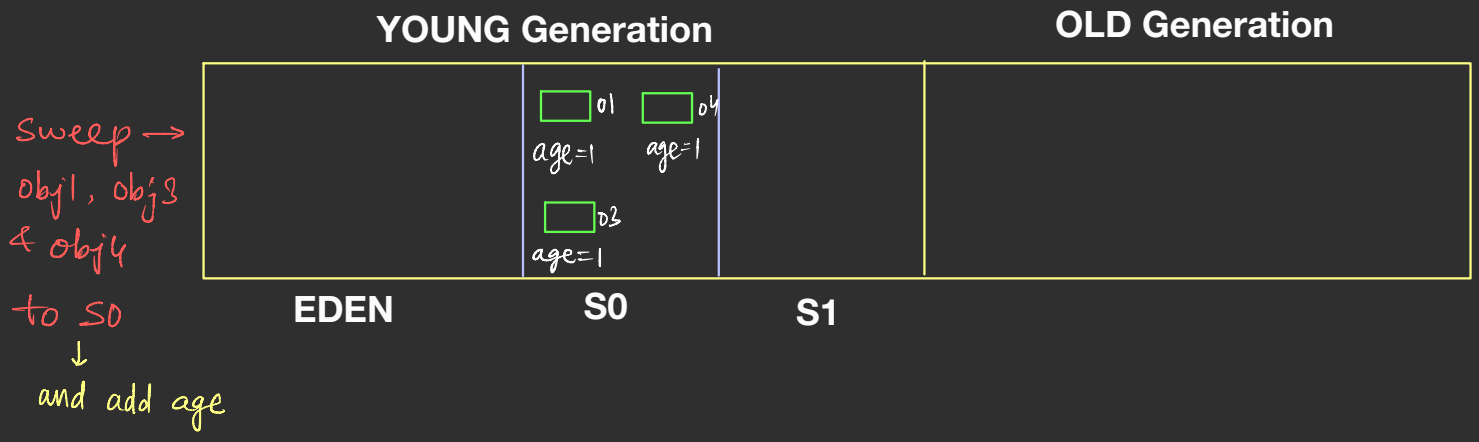
① GC invokes
↓
Mark & sweep Algo
↳ It will mark → all the objects which are no more referenced & it is allowed to delete

mark

→ let's say at that time → obj 2 & obj 5 has no reference & allowed to delete.

sweep → sweep other surviving objects to either S0 or S1.



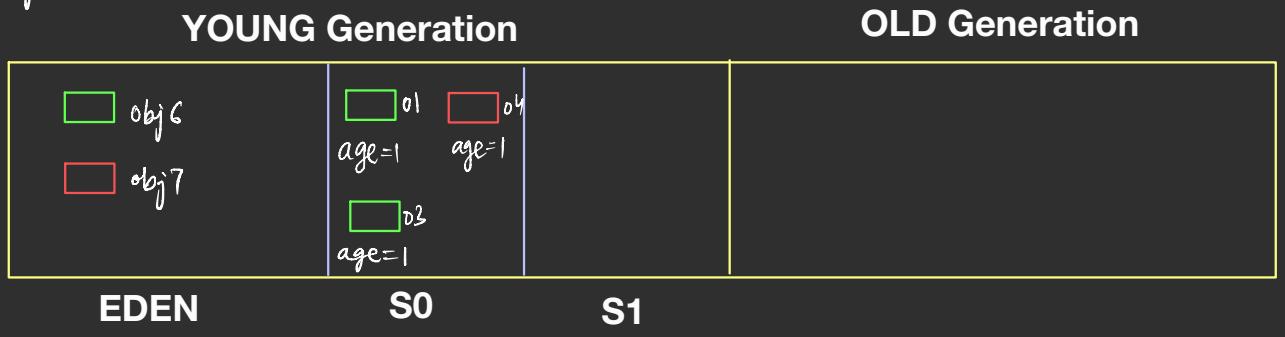


This is called "Minor GC" ⇒ because it is very fast ↓ happen very periodically

Now let's say, before any GC runs another objects created.

obj 6

obj 7



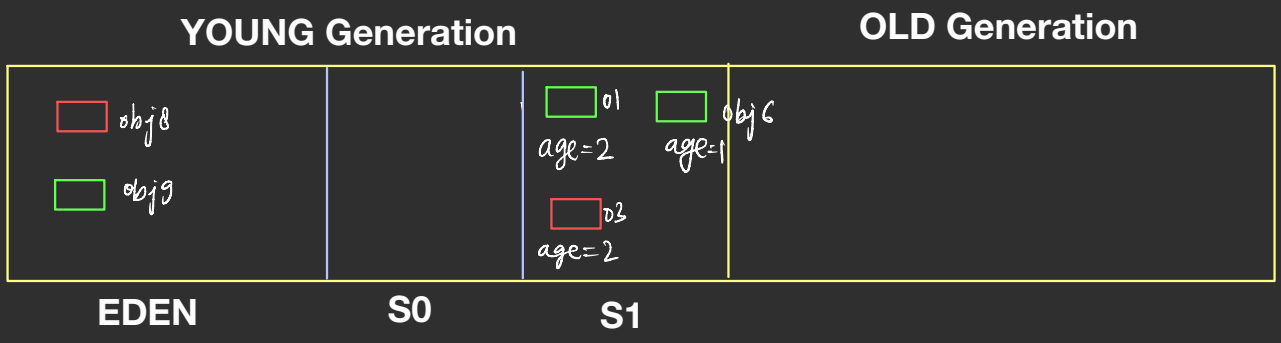
② GC Invokes

mark → let's say obj 7 & obj 4 have no reference.

sweep → delete obj 7 & obj 4 & put all other in S1 & increase their age.

↓

EDEN and one of the S0 or S1 will be free after GC



Now let's say, before any GC runs another objects created.

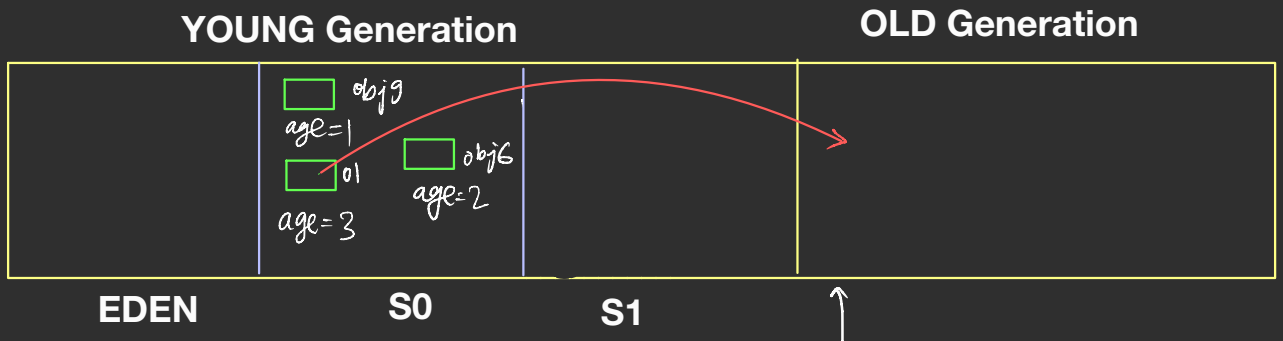
obj0

obj9

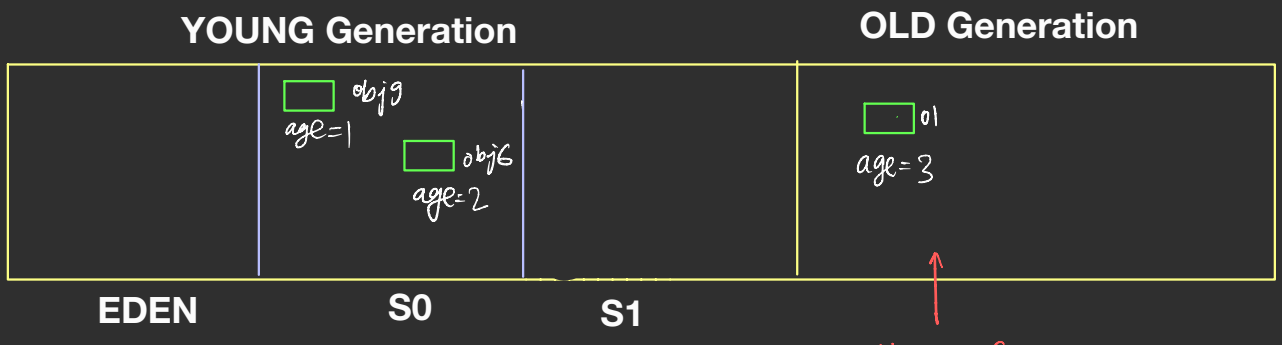
③ GC invokes 3rd time — Threshold age = 3 ⟹ Promotion to Old Generation

mark → let's say obj8 & obj3 have no reference.

sweep → delete obj8 & obj3 & put all other in S0 & increase their age



This is called "Major GC"



Here GC runs very less Periodically

Metaspace

↳ Outside of the Heap

- Stores → class-variables
  - ↓ static
- → Class-metadata
  - ↓ Information about the class from which class-object can be created.
- → Constanst
  - ↳ static final

• Garbage Collector

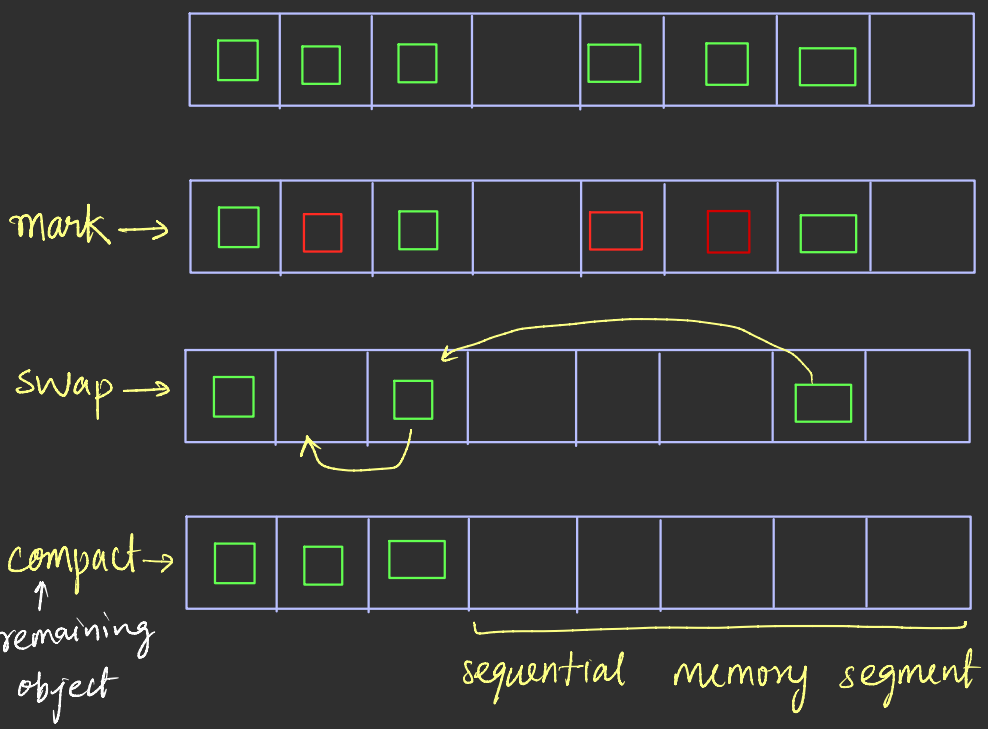
Mark and Sweep Algorithm

↓

mark → identify what all objects needs to be deleted.

sweep → Remove those objects from the memory.

Mark and Swap with compact memory



TOC

• Versions of Garbage Collector

① Serial GC

↑

Only 1 thread use in minor as well as major GC.

↓

Slow → All application thread will pause if GC runs

② Parallel GC → default in java 8

↓

multiple thread working on GC

③ Concurrent Mark & sweep (CMS)

↓

Application threads does not stops during GC threads run.

↓

But not 100% guarantee → JVM try

④ G1 GC → Best

Application thread will not stop, also brings compaction

↓

all allocated memory together & free are together

↳ In further Java there are chances to reduce pause time

Throughput ↑

Latency ↓

TOC

🔗 Click Here for HOME PAGE

🔗 For Classes in Java Follow this Link